PROOF-OF-CONCEPT — CLIENT ENGAGEMENT REPORT

AI-Assisted QA for Web Accessibility

State-of-the-art WCAG compliance, shifted left into development

How we define the problem, the solution we propose, the exact POC lifecycle, the detailed build steps, the evaluation plan, and an honest account of what this POC will and will not achieve — prepared for review by the client's technical leadership.

Scope: Pragmatic shift-left QA + one SOTA AI layer (phaseable to frontier)

Prepared by: Senior AI/ML Engineering Team, Edvensoft
Prepared for: Client Technical Leadership (expert AI reviewer)
Engagement: AI-Assisted QA for Web Accessibility — POC definition & build plan
Date: August 2026 • **Classification:** Confidential

1. Executive summary

You asked us to build a proof-of-concept for **AI-assisted QA that reduces web-accessibility (WCAG) issues introduced during development** — and to make it state-of-the-art, not just another scanner. This report lays out, end to end, how we define that problem, the solution we propose, exactly how we will build the POC, how we will measure whether it worked, and where the honest limits are.

The core insight driving our approach is simple and well-evidenced: **traditional rule-based accessibility scanners only catch roughly a third of WCAG issues by criteria** (about 16 of ~50 WCAG 2.1 AA success criteria are machine-testable), because they can verify *syntax* ("is an `alt` attribute present?") but cannot judge *meaning* ("is that alt text actually useful?"). Everything else today falls to slow, expensive manual audits that happen too late. Our POC closes a large part of that gap by adding a **grounded AI semantic-and-visual layer** on top of a rock-solid deterministic core, and by delivering it **where issues are actually born — inside the developer's pull request, before code merges.**

> **What we will demonstrate in the POC (≈6–8 weeks):** a working system that, on every pull request against the client's web app, automatically (1) runs the deterministic rule engine, (2) runs an AI layer that judges the "meaning" issues rules miss — is this alt text meaningful, is this really a heading/button, does color alone carry meaning, is this label correct — each with a **confidence score and a cited WCAG reason**, and (3) posts a clear, actionable review comment and an optional merge gate. We will benchmark it against a human-audited "golden set" and report precision/recall, false-positive rate, and issues-caught-before-merge.

What we are deliberately honest about: no AI system today can certify full WCAG conformance on its own, and we will not claim it can — a position reinforced by a 2025 U.S. FTC action that fined an accessibility-automation vendor $1M for exactly that kind of over-claim. Our POC is designed to **catch far more, far earlier, with far less human effort** — and to route the genuinely subjective residue to humans efficiently — not to replace expert judgment. Section 9 (Pros/Cons) and Section 10 (Grey Areas) are explicit about this.

The five questions this report answers

Your question	Where we answer it
How do we approach & define the problem?	§2 Understanding the requirement, §3 Problem definition
How do we define the solution?	§4 Solution definition, §5 Reference architecture (concrete stack)
What are the exact POC lifecycle steps?	§6 POC lifecycle
How exactly do we build it?	§7 Detailed build steps, §8 Evaluation & KPIs
Pros, cons, and what's in the grey area?	§9 Pros & cons, §10 Grey areas

(Plus) Timeline, team, cost, risks, next steps	§11 Delivery plan, §12 Risks, §13 Path to production

2. Understanding the client requirement

Before defining a solution we restated the requirement in our own words and pressure-tested what "success" really means, because the phrasing carries three specific engineering commitments that shape everything downstream.

What the client actually said

"A POC solution on AI-Assisted QA for Web Accessibility which is SOTA in WCAG guidelines & compliance, where the core idea is to reduce web-accessibility issues introduced during development of a product/ platform/app."

What that implies — reading the requirement precisely

Phrase in the brief	How we interpret it (the engineering commitment)
"QA"	This is a *quality-assurance* system, not an end-user assistive tool and not a runtime overlay. It sits in the engineering workflow and produces findings developers act on.
"reduce issues introduced *during development*"	The decisive phrase. It mandates a **"shift-left"** design: the system must operate at commit / pull-request / CI time, so defects are caught *before* they reach staging or production — where they are 10–100× cheaper to fix. A post-launch scanner would not satisfy this.
"SOTA"	It must do meaningfully more than the free rule-based scanners the client's team could already install. The differentiator is an **AI layer that judges semantic and visual correctness** — the ~two-thirds of WCAG that deterministic rules cannot decide — with grounding and confidence, not black-box guesses.
"WCAG guidelines & compliance"	Findings must map to specific, citable **WCAG 2.2 success criteria** (the current standard) at Level A/AA. Every issue the system raises must say *which* criterion it relates to and *why* — so it is defensible and educational, not just a red flag.
"POC"	The goal is to *prove the core hypothesis* on the client's real code with measurable evidence — not to ship a hardened product. Scope discipline and a crisp success bar matter more than feature breadth.

> **The hypothesis we are testing (stated so it can pass or fail):** "An AI layer, grounded in WCAG and integrated into the client's pull-request workflow, can catch a materially larger share of real accessibility defects than a rule engine alone — *before merge*, at an acceptable false-positive rate, and with developer-usable explanations." Section 8 defines the exact numbers that constitute "pass."

Who the stakeholders are (and what each needs from the POC)

- **Developers** — need accurate, low-noise findings inside their existing tools (PR, IDE), with a suggested fix. Noise kills adoption faster than anything.
- **QA / accessibility leads** — need coverage they can trust and a way to see what still needs human review.

- **Engineering leadership** — needs proof that defects drop and that the merge gate won't block delivery with false alarms.
- **Legal / compliance** — needs findings tied to WCAG criteria and an honest statement of residual risk (no false "we're fully compliant" claim).

3. Problem definition

Here we define the problem formally — what makes it hard, why the current state fails, and the precise gap the POC targets.

3.1 The current state and why it fails

Accessibility defects are introduced continuously during development — a missing label here, a `<div>` used as a button there, an icon with no accessible name, a color-only status indicator. Today teams find these in one of two ways, both flawed for "during development":

- **Rule-based scanners** (axe-core, Lighthouse, WAVE) run fast and deterministically, but are structurally limited: they reliably decide only the machine-checkable criteria. Industry measurement puts this at roughly **a third of WCAG 2.1 AA criteria (~16 of ~50)**; by raw issue *volume* Deque measures automation at ~57% because a few auto-detectable issues (like contrast) are very frequent. Either way, the criteria that most affect real users — meaningful alt text, correct reading order, real vs. fake semantics, sensible labels — need judgment the rules do not have.
- **Manual expert audits** catch the rest but are slow, expensive, and happen *late* (pre-launch or, worse, after a complaint). By then the defect is deep in the codebase and costly to unwind.

> **The core problem in one sentence:** the cheap, fast check (rules) is too shallow, and the deep check (humans) is too slow and too late — so accessibility debt accumulates silently during development and surfaces expensively at the end.

3.2 What makes this genuinely hard (the technical crux)

- **"Meaning" is not in the markup.** Whether `alt="image123"` is *useful* depends on the image content and the page context, not the HTML. Deciding it requires understanding pixels and intent — a multimodal problem.
- **The rendered UI diverges from the code.** A control can look and behave like a button to a sighted user while being an unlabeled `<div>` to a screen reader. Catching this needs a view of the *rendered, accessibility-tree* reality, not just static source.
- **Dynamic, stateful apps.** Modern React/Angular/Vue apps change the DOM after interaction (modals, menus, route changes). Many defects only exist in a particular state, so testing must exercise states, not just load a page.
- **False positives are poison.** A QA tool that cries wolf gets switched off. Precision and clear explanations matter as much as recall.
- **Non-determinism & hallucination.** The moment we add an LLM/VLM, outputs become probabilistic and can be confidently wrong — unacceptable without grounding, confidence scoring, and a human path for the uncertain residue.

3.3 The gap the POC targets (scoping the problem down)

We are *not* trying to boil the ocean in the POC. We target the highest-value, most tractable slice of the "semantic gap":

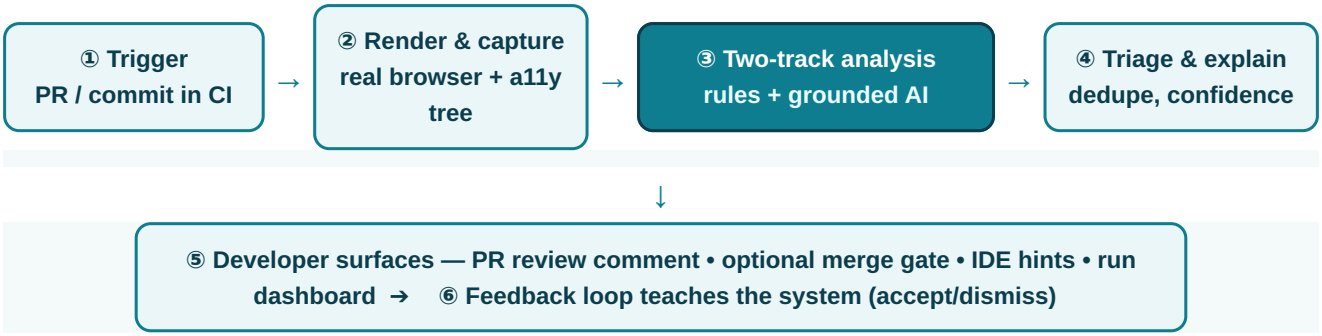
In scope for the POC	Deferred (Phase 2 / out of scope)
AI judgment of **meaningful alt text** (image + page context)	Full agentic screen-reader emulation of entire user journeys
AI detection of **fake semantics** (div-as-button, fake headings, missing accessible names)	Automatic code-fix / self-healing pull requests
AI check of **color-only meaning** & visual/label sense on rendered pages	Cognitive-accessibility & plain-language judgments
Deterministic rule engine as the trusted base (contrast, ARIA validity, etc.)	PDF/document remediation, sign-language, media captioning
Integration into **one CI/PR workflow** on the client's real app	Multi-repo, org-wide rollout & governance dashboards

Rationale: this slice directly attacks the criteria rules can't judge, is demonstrable on real code in the POC window, and de-risks the frontier features by proving the foundation first. The deferred items are exactly the phase-2 path in §13.

4. Solution definition

Our solution is a **hybrid, grounded, shift-left accessibility QA system**. “Hybrid” = deterministic rules for what machines can prove + an AI layer for what needs judgment. “Grounded” = every AI decision is anchored to WCAG criteria and returns a confidence and a reason. “Shift-left” = it runs in the pull request, before merge.

4.1 The solution in one picture (the mental model)



4.2 The three design principles (non-negotiables)

1. **Deterministic core, AI on top — never AI alone.** The rule engine (axe-core) remains the trusted, reproducible base that catches the certain issues with zero variance. The AI layer only *adds* the judgment issues; it never overrides a hard rule. This keeps the system trustworthy and the AI’s scope bounded.
2. **Grounding + confidence + citation on every AI finding.** The AI is given the specific WCAG success criterion and ACT-rule context it must reason against (retrieval-augmented), and must return *{verdict, confidence 0–1, cited criterion, one-line rationale, suggested fix}*. Low-confidence items are surfaced as “needs human review,” not asserted as failures. This is how we control hallucination and keep false positives down.
3. **Meet developers where they are.** Findings appear in the pull request and IDE, in seconds-to-minutes, with a concrete fix. Adoption is a product problem as much as an ML one; a precise, quiet, well-explained tool gets kept on.

4.3 Why this is genuinely SOTA (and not just axe-core in a trench coat)

- It uses a **vision-language model to reason over the rendered page + accessibility tree**, targeting the specific criteria rules cannot decide — the same class of technique the leading commercial engines (Deque’s ML/CV layer, Evinced’s structural-semantic model) have moved toward, but assembled and grounded for *pre-merge* use.
- It is **retrieval-grounded in WCAG/ACT rules** rather than free-associating, which is the current best practice for reducing LLM hallucination in a compliance context.
- It ships as a **developer-time gate with confidence-scored, criterion-cited findings** — the shift-left delivery that turns “audit later” into “prevent now.”
- It is architected so the frontier capabilities (agentic AT-emulation testing, auto-fix PRs) plug into the *same* pipeline in Phase 2 — we prove the foundation now and extend, rather than betting the POC on research-grade features.

Build-vs-buy honesty (for an expert reviewer): mature engines exist (Deque axe DevTools, Evinced, Siteimprove) and we will benchmark against axe-core as our baseline. The POC's value is not re-inventing rule detection — it is proving a *grounded, pre-merge AI semantic layer tuned to the client's app and workflow*, and giving the client an owned, extensible foundation rather than a closed SaaS. We will present a clear build-vs-buy recommendation at the readout.

5. Reference architecture (concrete SOTA stack)

Below is the concrete stack we propose for the POC. Every component names a specific technology and a swappable alternative, so the client can standardize on their preferred cloud/model without changing the design.

5.1 The pipeline, layer by layer

① TRIGGER & ORCHESTRATION

GitHub Actions / GitLab CI | runs on pull_request | Docker job runner | alt: Jenkins, CircleCI, Azure Pipelines

Fires on every PR (and nightly on main). Spins up the analysis container, collects results, posts back to the PR, sets the status check.

② RENDER & CAPTURE (SEE THE APP AS A USER DOES)

Playwright + headless Chromium | DOM snapshot | Accessibility tree (CDP) | full-page + element screenshots | state driver | alt: Puppeteer, Selenium

Playwright renders the changed routes/components (ideally against the PR's preview deploy or a Storybook build), captures the DOM, the **Chrome accessibility tree** (what assistive tech actually sees), and screenshots. A lightweight **state driver** exercises key interactions (open modal, expand menu) so stateful defects are visible. This dual capture — code-truth (DOM/a11y tree) + visual-truth (pixels) — is what lets the AI compare "what a screen reader hears" vs. "what a sighted user sees."

③A DETERMINISTIC RULE ENGINE (THE TRUSTED BASE)

axe-core | WCAG 2.2 A/AA ruleset | zero-variance | needs-review flags | alt: IBM Equal Access, Pa11y

Runs first. Catches the machine-provable issues (contrast, ARIA validity, missing attributes) reliably and cheaply, and emits its own "needs review" items — which become candidate inputs for the AI layer.

③B GROUNDED AI SEMANTIC & VISUAL LAYER THE SOTA DIFFERENTIATOR

VLM: Claude / GPT-4o / Gemini (vision) | WCAG+ACT knowledge base | RAG (vector store: pgvector / FAISS) | structured JSON output | confidence scoring | alt: open VLM (Qwen-VL / Llama-Vision) self-hosted

The heart of the system. For each candidate element it builds a compact prompt from the *screenshot region + a11y-tree node + surrounding DOM/context*, retrieves the relevant WCAG success criterion and ACT-rule wording, and asks the VLM to return a strict JSON verdict: *{criterion, pass/fail/uncertain, confidence, one-line reason, suggested fix}*. Specialized "checkers" (meaningful-alt, real-vs-fake-semantics, color-only-meaning, label-correctness) are separate prompt/schema pairs so each is independently tunable and testable.

④ TRIAGE, DEDUPE & CONFIDENCE GATING

merge rules+AI findings | dedupe by element/criterion | confidence thresholds | severity = user-impact × WCAG level | suppress known-safe

Combines both tracks, removes duplicates, and applies thresholds: high-confidence → report as a failure; mid-confidence → report as "review"; low-confidence → drop (logged for tuning). Prioritizes by user impact and WCAG level (A before AA). This layer is where we buy down false positives.

⑤ DEVELOPER SURFACES

PR review comments (inline) | CI status check / merge gate | HTML/SARIF report artifact | IDE hints (VS Code) | optional dashboard

Findings appear as inline PR comments tied to the changed lines, plus a summarized status check that can (optionally) gate merge on new Level-A failures only — deliberately lenient at first to protect adoption. Results also export as **SARIF** so they show up natively in the code host's security/quality tab.

⑥ FEEDBACK & EVALUATION LOOP

accept / dismiss captured | golden-set benchmark | prompt & threshold tuning | telemetry | eval harness in CI

Every developer accept/dismiss is logged and feeds an offline evaluation harness that scores the system against a human-labeled golden set, so we can tune prompts and thresholds with evidence and detect regressions when a model or prompt changes.

5.2 Cross-cutting concerns (an expert will ask about these)

Concern	How the design handles it
Latency & cost	The AI layer only runs on *changed* components/routes (diff-aware), and only on axe-flagged or heuristically-selected candidate elements — not the whole app. Screenshots are region-cropped. Results are cached by content hash so unchanged elements aren't re-analyzed. Target: minutes per PR, cents-to-low-dollars per PR.
Hallucination	RAG grounding to exact WCAG/ACT text; strict JSON schema with a required "evidence" field; confidence gating; and an "uncertain→human" path. We measure hallucination directly via the golden set (false-positive rate is a headline KPI).
Determinism / reproducibility	Rule findings are fully deterministic. AI findings are made *near*-deterministic via temperature 0, pinned model versions, and cached verdicts; the report records model+prompt version for auditability.
Privacy / IP	Option to run against a self-hosted open VLM (Qwen-VL / Llama-Vision) or a no-retention enterprise API tier, so the client's UI/screenshots never train a third-party model. Decided at kickoff based on client policy.
Framework-agnostic	Because analysis is on the *rendered* DOM/a11y-tree/pixels, it works regardless of React/Angular/Vue/server-rendered — no framework coupling.

Model choice, stated plainly: for the POC we default to a **hosted frontier VLM** (Claude or GPT-4o class) for best judgment quality with least setup, behind a thin *model-gateway* abstraction so we can swap to a self-hosted open VLM for cost/privacy without touching the rest of the pipeline. The gateway also lets us route cheap/easy checks to a smaller model and hard ones to the frontier model.

6. POC lifecycle — the exact steps we lay out

This is the lifecycle our team runs for a POC of this type. It is deliberately evidence-driven: a defined hypothesis, a labeled ground truth to measure against, a build, a measurement, and a go/no-go decision.

#	Phase	What happens & exit criterion
0	**Discovery** Scoping & alignment	Confirm the hypothesis, target app/routes, WCAG target (2.2 AA), the specific AI checks in scope, model/privacy constraints, and success metrics. *Exit:* signed one-page POC charter.
1	**Ground truth** Golden-set creation	Assemble a representative sample of the client's pages/components and have an accessibility expert label them against WCAG (the "golden set"). This is what we measure the AI against. *Exit:* ~150–300 labeled elements/pages with adjudicated verdicts.
2	**Baseline** Deterministic core	Stand up render/capture + axe-core in CI; measure the baseline (what rules alone catch on the golden set). *Exit:* reproducible baseline numbers.
3	**SOTA build** AI layer	Build the grounded AI checkers (meaningful-alt, fake-semantics, color-only, labels), RAG grounding, JSON schema, confidence gating. Iterate against the golden set. *Exit:* AI layer beats baseline on recall at target precision.
4	**Integrate** Developer workflow	Wire triage + PR comments + status check + SARIF + optional IDE hints. Dogfood on internal PRs. *Exit:* a real PR shows accurate, actionable findings end-to-end.
5	**Pilot** On the client's app	Run on a live feature branch / set of real PRs for 1–2 weeks; capture developer accept/dismiss feedback; tune thresholds. *Exit:* KPIs measured on real traffic.
6	**Readout** Evaluate & decide	Score against KPIs, quantify issues-caught-before-merge and effort saved, present pros/cons/grey-areas and a build-vs-buy + productionization recommendation. *Exit:* go/no-go decision with evidence.

> **Why a golden set is step 1, not an afterthought:** for an AI QA system, "does it work?" is only answerable against human-labeled ground truth. Building it first forces a precise definition of success, gives us an objective tuning target, and produces the exact evidence the client's expert will want at the readout. Skipping it is the most common way these POCs become unfalsifiable demos.

7. How we build it — detailed, accurate steps

This is the concrete engineering plan mapped onto the ~6–8-week POC. Each week lists the work, the tech, and the deliverable. Weeks overlap in practice; they are shown sequentially for clarity.

Week 0–1 Foundation: capture harness + baseline in CI

- **Set up the CI job:** a Dockerized GitHub Actions workflow triggered on `pull_request`, running headless Chromium via Playwright against the PR's preview deploy (or a Storybook/component build if no preview exists).
- **Build the capture module:** for each target route/component, snapshot the *DOM*, the *accessibility tree* (via Chrome DevTools Protocol), and *screenshots* (full page + per-candidate-element crops). Add a small **state driver** to trigger key interactions (open modal, expand nav) and re-capture.
- **Integrate axe-core** and produce a normalized finding schema (element locator, criterion, severity, source=rule).
- **Deliverable:** a PR that runs, captures, runs axe-core, and posts a comment with rule findings. Baseline established.

Week 1–2 Ground truth: the golden set

- **Select a representative sample** of the client's UI (forms, tables, nav, modals, media, data views) — the patterns that generate most real defects.
- **Expert labeling:** an accessibility specialist (ours or the client's) labels each element against the in-scope WCAG 2.2 criteria with a verdict + rationale; a second reviewer adjudicates disagreements. Store as versioned JSON.
- **Deliverable:** a 150–300-item golden set with adjudicated labels and an automated scoring script — the yardstick for everything that follows.

Week 2–4 The SOTA AI layer

- **Model gateway:** a thin abstraction (LiteLLM-style) so any VLM (Claude/GPT-4o/Gemini, or a self-hosted open VLM) is callable behind one interface, at temperature 0, with pinned versions.
- **WCAG/ACT knowledge base + RAG:** ingest the WCAG 2.2 success criteria, Understanding docs, and ACT rules into a vector store (pgvector/FAISS); at analysis time, retrieve the exact criterion text for the check being run and inject it into the prompt as grounding.
- **Build the checkers** as independent prompt+schema pairs, each returning strict JSON *{criterion, verdict, confidence, evidence, suggested_fix}*:
  - **Meaningful-alt checker** — input: image crop + existing alt + surrounding text; judges whether the alt conveys the image's purpose *in context* (and flags decorative-vs-informative mistakes).
  - **Real-vs-fake-semantics checker** — input: a11y-tree node + screenshot + DOM; flags "looks/behaves like a control but isn't exposed as one" (div-as-button, fake heading, icon-button with no name).
  - **Color-only-meaning checker** — input: screenshot region + text; flags information conveyed by color alone (e.g., red = required) with no text/'*' alternative.
  - **Label-correctness checker** — input: form control + visible label + programmatic name; flags mismatches and placeholder-as-label.

- **Confidence & gating:** calibrate thresholds on the golden set; wire the "uncertain → needs human review" path.
- **Deliverable:** AI layer scoring measurably above baseline recall at the agreed precision, on the golden set, with a reproducible eval report.

Week 4–5 Triage + developer surfaces

- **Triage engine:** merge rule + AI findings, dedupe by element×criterion, apply confidence gates, compute severity, suppress an allow-list of known-safe patterns.
- **Surfaces:** inline PR review comments (mapped to changed files/lines), a summarizing **status check** (configurable merge gate — start with "gate only on *new* Level-A failures"), a downloadable HTML report, and **SARIF** export for the code host's native quality tab. Optional VS Code hints.
- **Deliverable:** a real internal PR that shows accurate, well-explained, low-noise findings with suggested fixes.

Week 5–7 Pilot on the client's app + tuning

- Run on a set of live feature branches for 1–2 weeks. Capture **accept/dismiss** telemetry from real developers.
- Tune prompts/thresholds against real feedback; drive the false-positive rate down; expand the allow-list.
- **Deliverable:** KPIs measured on real PRs; a tuned, stable configuration.

Week 7–8 Readout & recommendation

- Compile the evaluation, the issues-caught-before-merge count, estimated effort saved, pros/cons/grey-areas, and a productionization + build-vs-buy recommendation with a Phase-2 plan.
- **Deliverable:** this style of final report + a live demo + the code/repo handover.

> **What the client's team needs to provide** (so there are no surprises): access to the app's repo and a way to render PRs (preview deploys or a component build), 1 accessibility SME for ~part-time labeling/adjudication, a CI environment or permission to add our workflow, and a decision on the model/privacy option (hosted vs self-hosted).

8. Evaluation & success metrics (how we prove it worked)

A POC is only credible if it is measured. These are the KPIs we commit to reporting, all computed against the human-labeled golden set and the live pilot.

Recall ↑

% of real in-scope defects the system catches vs. golden set. Headline: AI layer must beat axe-core baseline by a wide margin on the semantic criteria.

Precision ↑

% of raised findings that are true defects. Target a high bar (e.g., ≥85% on high-confidence findings) — low noise protects adoption.

FP rate ↓

False positives per PR / per 100 findings. A primary go/no-go metric; directly measures hallucination.

Pre-merge catch ↑

% of injected/known defects caught *before* merge — the direct measure of "reduce issues introduced during development."

Effort saved ↑

Estimated manual-audit hours avoided, from criteria auto-cleared × typical review time.

Latency & cost

Median minutes per PR and $ per PR — must be workflow-acceptable.

Evaluation method

- **Offline benchmark:** run the full pipeline over the golden set; compute per-criterion precision/recall/F1 for rules-only, AI-only, and hybrid; produce a confusion matrix per checker. This isolates exactly where the AI adds value.
- **Injected-defect test:** deliberately introduce a catalogue of known accessibility regressions into test PRs and measure the catch rate before merge — a clean, repeatable proxy for "defects prevented."
- **Live pilot:** on real PRs, track accept/dismiss, developer-reported usefulness, and noise. This tests the thing offline metrics can't: whether developers actually keep it on.
- **Human cross-check:** an accessibility SME reviews a sample of the system's "pass" verdicts too, to estimate *missed* defects (false negatives), not just false positives.

> **The single sentence that defines POC success:** "On the client's real app, the hybrid system catches a substantial majority of in-scope semantic defects *before merge*, at a false-positive rate developers tolerate, with findings they act on — measurably beating the rule engine alone." If we hit that, it's a go; if the false-positive rate can't be tamed to an acceptable level, that is a documented, honest no-go on that check — and we say so.

9. Pros & cons of this POC

Pros / what it achieves

- **Directly hits the goal:** catches defects *during development*, pre-merge, where they're cheapest to fix.
- **Real SOTA lift:** attacks the ~two-thirds of WCAG criteria rule engines can't judge, via grounded VLM reasoning.
- **Low-noise by design:** confidence gating + grounding + "uncertain→human" keeps false positives down and trust up.
- **Explainable & educational:** every finding cites a WCAG criterion, gives a reason and a fix — developers learn as they go.
- **Framework-agnostic & portable:** works on rendered output; not tied to React/Angular/Vue or a vendor SaaS.
- **Owned & extensible:** the client gets an asset that phases cleanly toward agentic testing and auto-fix.
- **Measurable:** golden-set + injected-defect evaluation gives hard evidence, not vibes.

Cons / costs & constraints

- **Not full conformance:** it reduces and catches issues; it does not certify WCAG compliance (and we won't claim it does).
- **Needs a golden set:** requires expert labeling effort up front — real cost, but non-negotiable for credibility.
- **Per-PR cost/latency:** VLM calls add seconds-to-minutes and compute cost; mitigated by diff-scoping and caching, but non-zero.
- **Residual false positives/negatives:** AI judgment is probabilistic; some noise and some misses remain and must be managed.
- **Coverage boundaries:** keyboard-journey, cognitive, and full-page-flow criteria are only partially addressed in the POC scope.
- **Model/vendor dependency:** hosted-model choice brings cost, rate-limit, and data-policy considerations (mitigated by the gateway + self-host option).
- **Adoption risk:** value depends on developers keeping the gate on — a product/change-management effort, not just ML.

10. Grey areas — what may or may not be achievable

For an expert reviewer, the honest “maybe” column matters as much as the confident one. These are the areas where outcomes depend on the app, the data, and how far we push — we flag them openly rather than promising them.

Grey area	Our stance	Why it’s uncertain / how we’ll test it
Reliable “meaningful alt text” judgment	**Likely, with tuning**	VLMs are strong at this but can miss author intent and page purpose. The golden set will tell us the real precision/recall; we expect good-but-not-perfect, surfaced with confidence.
Catching *every* fake-semantics case	**Partial**	Common patterns (div-as-button) are very catchable; exotic custom widgets and complex ARIA compositions are harder and may need human review.
Full keyboard-operability & focus-order testing	**Phase 2**	Requires agentic interaction (tabbing through, trapping focus) — buildable but beyond the pragmatic POC; we’ll prototype-preview it, not productize it.
Cognitive / plain-language criteria	**Out of scope**	Highly subjective; WCAG itself is still maturing here (WCAG 3 direction). We won’t assert these in the POC.
Auto-fixing issues (self-healing PRs)	**Deferred**	Generating correct ARIA/code fixes is promising (cf. Deque Axe MCP, GitHub’s a11y agent) but risky to auto-apply; POC will *suggest* fixes, not commit them.
A defensible “WCAG compliant” claim	**No**	No automated system can certify conformance; manual + assistive-tech testing is still required. Over-claiming here has real legal risk (the 2025 FTC $1M action against an overlay vendor). We position the tool as risk-reduction, not certification.
Non-deterministic outputs in a compliance context	**Managed, not eliminated**	Temperature 0, pinned versions, and caching make it near-deterministic; we record model+prompt versions for auditability, but cannot claim bit-for-bit determinism from an LLM.

> **The one line to remember on grey areas:** the POC is engineered to *maximize what’s catchable early and be transparent about the rest*. Anything uncertain is surfaced as “needs review” with a confidence score — never silently asserted — so the system degrades gracefully and never misleads the client into a false sense of compliance.

11. Delivery plan — timeline, team, effort & cost

A complete POC plan for the client's planning purposes. Effort and cost are illustrative engineering estimates for a POC of this scope; they firm up after the Discovery phase.

11.1 Timeline (≈8 weeks)

Weeks	Phase	Primary output
0–1	Discovery + capture harness & baseline	Charter; CI job renders PRs, runs axe-core, posts comments
1–2	Golden-set creation	150–300 expert-labeled items + scoring script
2–4	AI semantic/visual layer	Grounded checkers beating baseline on golden set
4–5	Triage + developer surfaces	PR comments, merge gate, SARIF, IDE hints
5–7	Pilot on client app + tuning	KPIs on real PRs; tuned config
7–8	Readout & recommendation	Final report + demo + repo handover + Phase-2 plan

11.2 Team & roles

Role	Allocation	Responsibility
ML/AI Engineer (lead)	Full-time	AI checkers, RAG grounding, model gateway, evaluation harness
Full-stack / DevOps Engineer	Full-time	Playwright capture, CI integration, PR surfaces, SARIF, dashboard
Accessibility SME	Part-time (~40%)	Golden-set labeling/adjudication, WCAG mapping, validating findings
Tech Lead / Architect	Part-time (~25%)	Architecture, thresholds strategy, build-vs-buy, readout
Project/Delivery Lead	Part-time (~15%)	Client comms, charter, scope control, readout logistics

Lean by design: ~2 full-time engineers plus fractional SME/architect/PM. This is the right shape for a POC — small, senior, fast.

11.3 Effort & cost (illustrative)

Item	POC estimate
Engineering effort	~2.7 FTE × 8 weeks ≈ **18–22 person-weeks**

Model / inference cost (POC)	Low — hundreds of dollars range (diff-scoped calls, caching); firmer after Week 2
Infra (CI runners, vector store, preview envs)	Modest; mostly existing client CI + a small container
One-time golden-set labeling	Bundled in SME allocation (~1–2 weeks part-time)

We intentionally quote effort in person-weeks rather than a fixed price here; a firm commercial quote follows the Discovery charter, when app scope and model/privacy choices are fixed.

12. Risk register & mitigations

Risk	Likely	Impact	Mitigation
False-positive rate too high → developers distrust it	Med	High	Confidence gating, grounding, “uncertain→human,” conservative gate (Level-A only), allow-lists, live tuning against accept/dismiss.
No PR preview environment to render against	Med	Med	Fall back to Storybook/component build or ephemeral deploy; decided in Discovery.
VLM cost/latency unacceptable at scale	Low–Med	Med	Diff-scoping, region crops, content-hash caching, route easy checks to smaller models, self-host option.
Model/data-privacy policy blocks hosted APIs	Med	Med	Model gateway + self-hosted open VLM (Qwen-VL/Llama-Vision) or no-retention enterprise tier; chosen at kickoff.
Golden set unrepresentative → misleading metrics	Low	High	Sample the highest-defect UI patterns; SME adjudication; expand set during pilot.
Hallucinated findings on edge cases	Med	Med	Strict JSON schema + required evidence field; measured directly via FP rate; low-confidence dropped.
Scope creep toward frontier features	Med	Med	Charter locks POC scope; frontier items explicitly routed to Phase 2 (§13).
Over-claim risk (legal)	Low	High	Explicit “risk-reduction, not certification” positioning throughout; no compliance guarantee.

13. Path to production & the Phase-2 frontier

The POC is architected so success extends cleanly into a product. Here is the honest roadmap from "proven" to "productionized," including the ambitious features we deliberately deferred.

From POC to production (hardening)

- **Coverage:** broaden from sampled routes to the whole app and multiple repos; expand the checker set; org-wide policy for the merge gate.
- **Reliability & cost:** model routing/quantization, batch/off-peak analysis, caching at scale, SLOs on latency and cost per PR.
- **Governance:** a dashboard for trends and residual risk, VPAT/ACR support, audit logs of model+prompt versions, role-based access.
- **Human-in-the-loop console:** an efficient review queue for "uncertain" items, whose adjudications continuously improve the golden set and thresholds.

The Phase-2 frontier (the "Option B" features we deferred)

Frontier capability	What it adds & why it's Phase 2
Agentic AT-emulation testing	An LLM agent that navigates the app via the accessibility tree the way a screen-reader/keyboard user would, testing whole journeys (checkout, forms) for barriers a static scan can't see. High value, but needs reliability engineering before it gates merges.
VLM visual-regression QA	Diff the rendered UI across states/PRs to catch visual accessibility regressions (focus not visible, target too small, reflow breakage) at pixel level.
AI auto-fix / self-healing PRs	Beyond suggesting fixes, generate the ARIA/semantic code change and open a fix PR for human approval (grounded like Deque Axe MCP / GitHub's a11y agent). Deferred because auto-applied fixes need strong guardrails.
Design-time shift (even further left)	Extend the same grounded checks into Figma so issues are caught before code exists.

Our recommendation

> Proceed with the **Scope-A POC as specified**: it is buildable and demoable in ~8 weeks, it directly proves the client's core hypothesis with hard numbers, and it produces an owned, extensible foundation. Run the golden-set-driven evaluation, make the go/no-go on evidence, and — on a go — harden into production while layering in the Phase-2 frontier capabilities one at a time, each earning its place against the same measurement bar. This gives the client a state-of-the-art capability without betting the first milestone on research-grade features.

Appendix A — Selected references & prior art

- W3C/WAI — WCAG 2.2 (current standard) and ACT Rules: w3.org/WAI/standards-guidelines/wcag; ACT Rules

- W3C/WAI — “no tool alone can determine conformance” (evaluation): w3.org/WAI/test-evaluate
- Deque — Automated Accessibility Coverage Report (~57% by volume; ~16/50 criteria machine-testable): deque.com/automated-accessibility-coverage-report
- Deque — Axe MCP Server (AI code-fix prior art): deque.com/axe/mcp-server
- Evinced — computer-vision structural-semantic detection: evinced.com/technology
- GitHub — general-purpose accessibility agent (auto-fix prior art, 68% resolution in pilot): github.blog/…/accessibility-agent
- FTC — $1M action against accessiBe for deceptive automated-compliance claims (why we don’t over-claim): ftc.gov/…/2025/01/…
- Tooling: Playwright (playwright.dev), axe-core (github.com/dequelabs/axe-core), SARIF (sarifweb).

This report defines and scopes the POC; exact metrics, costs, and the build-vs-buy call are finalized at the Discovery charter and the Week-8 readout. Companion document: “Utilization of AI to Manage Web Accessibility & WCAG Compliance” (strategic view).